```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

Cotporate Styling

```
plt.style.use('seaborn-v0_8')
sns.set_palette('husl')
print("Everything is fine")
```

```
Everything is fine
```

Load Dataset

```
df=pd.read_csv("/content/sample_data/funnel_analysis_data.csv")
df.head()
```

	User_ID	Session_ID	Stage	Timestamp	Device	Region	Channel	Revenue	Bounce_Flage	Bounce_Flag
0	1	e7648b11-8c61-43ec-b997-d308ce4f5f04	Visit	43:52.7	Tablet	South	Organic	0	Yes	NaN
1	1	a49a0e20-7db8-47a0-b2fc-76eec6a023aa	View Product	33:16.0	Tablet	East	Referral	0	Yes	NaN
2	2	4b001086-caa9-4bd9-9889-b5b2b27b25bd	Visit	23:21.7	Desktop	West	Organic	0	Yes	NaN

Next steps: Generate code with df | New interactive sheet

Perform Basic Steps

```
print("Top 5 data")
display(df.head())
```

```
Top 5 data
```

	User_ID	Session_ID	Stage	Timestamp	Device	Region	Channel	Revenue	Bounce_Flage	Bounce_Flag
0	1	e7648b11-8c61-43ec-b997-d308ce4f5f04	Visit	43:52.7	Tablet	South	Organic	0	Yes	NaN
1	1	a49a0e20-7db8-47a0-b2fc-76eec6a023aa	View Product	33:16.0	Tablet	East	Referral	0	Yes	NaN
2	2	4b001086-caa9-4bd9-9889-b5b2b27b25bd	Visit	23:21.7	Desktop	West	Organic	0	Yes	NaN

```
# Last 5 rows
df.tail()
```

	User_ID	Session_ID	Stage	Timestamp	Device	Region	Channel	Revenue	Bounce_Flage	Bounce_Flag
19987	9996	63b3a962-33cb-48d6-b288-787d2eef9446	Visit	50:18.4	Tablet	West	Referral	0	Yes	NaN
19988	9997	c63b2507-ffd8-432b-a151-fe8474c0776e	Visit	30:25.6	Mobile	North	Referral	0	Yes	NaN
19989	9998	a00193f6-8dc6-4cbf-9518-c0d487686246	Visit	09:43.3	Tablet	East	Referral	0	Yes	NaN
19990	9999	123ee695-aaca-4861-bb72-efca10958752	Visit	36:23.4	Mobile	North	Paid	0	Yes	NaN

```
# Random data form dataset
df.sample()
```

	User_ID	Session_ID	Stage	Timestamp	Device	Region	Channel	Revenue	Bounce_Flage	Bounce_Flag
3648	3648	7f51c85e-b130-4fe8-961c-	Visit	15:00.2	Mobile	South	Referral	0	Yes	NaN

```
#find aTotal columnns name and number of columns
print(df.columns)
print(f"\nTotal Number of columns {df.columns.nunique()}")
```

```
Index(['User_ID', 'Session_ID', 'Stage', 'Timestamp', 'Device', 'Region',
       'Channel', 'Revenue', 'Bounce_Flage', 'Bounce_Flag'],
      dtype='object')

Total Number of columns 10
```

```
# type of dataset
type(df)
```

```
pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None

/usr/local/lib/python3.12/dist-packages/pandas/core/frame.py
Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns).
Arithmetic operations align on both row and column labels. Can be
thought of as a dict-like container for Series objects. The primary
```

```
df.dtypes
```

	0
User_ID	int64
Session_ID	object
Stage	object
Timestamp	object
Device	object
Region	object
Channel	object
Revenue	int64
Bounce_Flage	object
Bounce_Flag	object

dtype: object

```
df['Timestamp'] = pd.to_datetime('00:' + df['Timestamp'], format='%H:%M:%S.%f')
print(df.dtypes)
```

```
User_ID                 int64
Session_ID             object
Stage                  object
Timestamp      datetime64[ns]
Device                 object
Region                 object
Channel                object
Revenue                 int64
Bounce_Flage           object
Bounce_Flag            object
dtype: object
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19992 entries, 0 to 19991
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
```

```
---  ------         --------------  -----
 0   User_ID        19992 non-null  int64
 1   Session_ID     19992 non-null  object
 2   Stage          19992 non-null  object
 3   Timestamp      19992 non-null  datetime64[ns]
 4   Device         19992 non-null  object
 5   Region         19992 non-null  object
 6   Channel        19992 non-null  object
 7   Revenue        19992 non-null  int64
 8   Bounce_Flage   19992 non-null  object
 9   Bounce_Flag    126 non-null    object
dtypes: datetime64[ns](1), int64(2), object(7)
memory usage: 1.5+ MB
```

```
df.describe()
```

	User_ID	Timestamp	Revenue
count	19992.000000	19992	19992.000000
mean	5000.385554	1900-01-01 00:29:48.591351296	3.256703
min	1.000000	1900-01-01 00:00:00.500000	0.000000
25%	2499.750000	1900-01-01 00:14:46.900000	0.000000
50%	5000.500000	1900-01-01 00:29:47.300000	0.000000
75%	7501.000000	1900-01-01 00:44:43.175000064	0.000000
max	10000.000000	1900-01-01 00:59:59.900000	1903.000000
std	2887.162096	NaN	64.713303

```
# imp info about all columns
df.describe(include="object")
```

	Session_ID	Stage	Device	Region	Channel	Bounce_Flage	Bounce_Flag
count	19992	19992	19992	19992	19992	19992	126
unique	19992	4	3	4	3	2	1
top	dc90f032-e8d5-4d7e-b6dc-19c3d16333f1	Visit	Mobile	West	Organic	Yes	No
freq	1	16927	6738	5035	6736	19929	126

```
df.shape
```

```
(19992, 10)
```

```
pd.set_option("display.max_rows", None)
```

Data Preprocessing And Cleaning

```
#  check for the null and duplicate values
print("n\---Finding Null Values----\n")
null_values=df.isnull().sum()
print(null_values)
```

```
n\---Finding Null Values----

User_ID             0
Session_ID          0
Stage               0
Timestamp           0
Device              0
Region              0
Channel             0
Revenue             0
Bounce_Flage        0
Bounce_Flag     19866
dtype: int64
<>:2: SyntaxWarning:

invalid escape sequence '\-'

<>:2: SyntaxWarning:
```

```
invalid escape sequence '\-'

/tmp/ipykernel_502/1647104682.py:2: SyntaxWarning:

invalid escape sequence '\-'
```

```
print("n\---Finding Duplicate Values----\n")
duplicate_values=df.duplicated().sum()
print("Total numbers of duplicate_values in the dataset is {duplicate_values}")
```

```
n\---Finding Duplicate Values----

Total numbers of duplicate_values in the dataset is {duplicate_values}
<>:1: SyntaxWarning:

invalid escape sequence '\-'

<>:1: SyntaxWarning:

invalid escape sequence '\-'

/tmp/ipykernel_502/3802451636.py:1: SyntaxWarning:

invalid escape sequence '\-'
```

```
print("n\---Total Unique Data----")
unique_data=df.nunique()
print(unique_data)
```

```
n\---Total Unique Data----
User_ID         10000
Session_ID      19992
Stage               4
Timestamp       15357
Device              3
Region              4
Channel             3
Revenue            63
Bounce_Flage        2
Bounce_Flag         1
dtype: int64
<>:1: SyntaxWarning:

invalid escape sequence '\-'

<>:1: SyntaxWarning:

invalid escape sequence '\-'

/tmp/ipykernel_502/2246486611.py:1: SyntaxWarning:

invalid escape sequence '\-'
```

```
df['Event_sequence']=df.groupby('Session_ID').cumcount()+1
```

```
df.head()
```

	User_ID	Session_ID	Stage	Timestamp	Device	Region	Channel	Revenue	Bounce_Flage	Bounce_Flag	Event_sequence
0	1	e7648b11-8c61-43ec-b997-d308ce4f5f04	Visit	1900-01-01 00:43:52.700	Tablet	South	Organic	0	Yes	NaN	1
1	1	a49a0e20-7db8-47a0-b2fc-76eec6a023aa	View Product	1900-01-01 00:33:16.000	Tablet	East	Referral	0	Yes	NaN	1
2	2	4b001086-caa9-4bd9-9889-b5b2b27b25bd	Visit	1900-01-01 00:23:21.700	Desktop	West	Organic	0	Yes	NaN	1

Next steps: Generate code with df | New interactive sheet

```
# Basics steps
print(f'Total User ID{df['User_ID'].nunique()}')
print(f'\nTotal Session ID{df['Session_ID'].nunique()}')
print(f'\nTotal Events{df['Stage'].nunique()}')
```

```
Total User ID10000

Total Session ID19992

Total Events4
```

Funnel Stage Definition and Session-Level Aggregation

```
# Define funnel stages in order

funnel_stages=['Browse', 'Add to Cart', 'Checkout', 'Purchase']

# Create session-Level summary
session_summary = df.groupby('Session_ID').agg(
    User_ID=('User_ID', 'first'),
    Session_Start=('Timestamp', 'min'), # Capture the start time of the session
    Session_End=('Timestamp', 'max'),   # Capture the end time of the session
    Events=('Stage', lambda x: x.tolist()),
    Device=('Device', 'first'),
    Region=('Region', 'first'),
    Channel=('Channel', 'first'),
    # Product_Category=('Product_Category', 'first'), # This column does not exist in the original dataframe
    Revenue=('Revenue', 'sum'),
    Bounce_Flag=('Bounce_Flag', 'first')
).reset_index()

# The previous line to flatten column names is no longer needed as named aggregations define them directly.
# session_summary.columns=['Session_ID', 'User_ID', 'Timestamp', 'Events',
#                          'Device', 'Region', 'Channel',
#                          'Product_Category', 'Revenue', 'Bounce_Flag']

#Calculate session duration in minutes
session_summary['Session_Duration_Min']=(session_summary['Session_End']-session_summary['Session_Start']).dt.total_sec
```

```
#identify max funnel stage reached for each session
def get_max_funnel_stage(events):
  stage_values={stage: i for i, stage in enumerate(funnel_stages)}
  max_stage_index=-1
  for event in events:
    # Ensure the event from the session_summary['Events'] list is in the defined funnel_stages
    if event in stage_values:
      max_stage_index=max(max_stage_index, stage_values[event])
  return funnel_stages[max_stage_index] if max_stage_index!= -1 else 'Browse'

# Apply the function to create the 'Max_Funnel_Stage' column
session_summary['Max_Funnel_Stage']=session_summary['Events'].apply(get_max_funnel_stage)
print("Session summary created")
display(session_summary.head())
```

```
Session summary created
```

	Session_ID	User_ID	Session_Start	Session_End	Events	Device	Region	Channel	Revenue	Bounce_Flag	Session_Duration_M
0	00020098-f66e-4215-9412-f91a7ec094fb	1401	1900-01-01 00:40:25.200	1900-01-01 00:40:25.200	[Visit]	Mobile	East	Referral	0	None	0
1	0002e574-fde7-4f01-9c63-4324e13ddeb5	5981	1900-01-01 00:15:30.700	1900-01-01 00:15:30.700	[View Product]	Tablet	East	Referral	0	None	0
2	0007a0d7-dd62-49f6-bc16-061b3b6e91aa	9660	1900-01-01 00:00:50.400	1900-01-01 00:00:50.400	[Visit]	Tablet	South	Referral	0	None	0
3	0007f123-c2aa-4c26-9cf8-d0aaaaa69f3d	8323	1900-01-01 00:52:14.600	1900-01-01 00:52:14.600	[View Product]	Mobile	West	Organic	0	None	0
4	000859eb-fb97-4c73-9d3f-cafb8386e676	1885	1900-01-01 00:37:29.100	1900-01-01 00:37:29.100	[Visit]	Desktop	West	Organic	0	None	0

Overall Funnel Analysis

```
# Calculate overall funnel metrics
funnel_metrics = []
for i, stage in enumerate(funnel_stages):
    if i == 0:
        # For Browse stage, count all sessions
        count = len(session_summary)
    else:
        # For other stages,count sessions that reached at least this stage
        count = len(session_summary[session_summary['Max_Funnel_Stage'].isin(funnel_stages[i:])])

    funnel_metrics.append({
        'Stage': stage,
        'Sessions': count,
        'Stage_Order': i
    })

# Create DataFrame from funnel metrics
funnel_df = pd.DataFrame(funnel_metrics)

#  Calculate conversion and drop-off rates
funnel_df['Conversion_Rate'] = (funnel_df['Sessions'] / funnel_df['Sessions'].iloc[0] * 100).round(2)
funnel_df['Drop_Off_Rate'] = (1 - funnel_df['Sessions'] / funnel_df['Sessions'].shift(1)) * 100
funnel_df['Drop_Off_Rate'].iloc[0] = 0
funnel_df['Drop_Off_Rate'] = funnel_df['Drop_Off_Rate'].round(2)

print("Overall Funnel Analysis:")
display(funnel_df)

#Revenue analysis
revenue_stats = session_summary[session_summary['Max_Funnel_Stage'] == 'Purchase'].agg({
    'Revenue': ['sum', 'mean', 'count']
}).round(2)

print("\nRevenue Analysis:")
print(f"Total Revenue: ${revenue_stats.iloc[0, 0]:.2f}")
print(f"Average Order Value: ${revenue_stats.iloc[1, 0]:.2f}")
print(f"Total Orders: {revenue_stats.iloc[2, 0]}")
```

```
Overall Funnel Analysis:
/tmp/ipykernel_502/1917152962.py:23: FutureWarning:

ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which wi
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver


/tmp/ipykernel_502/1917152962.py:23: SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver
```

	Stage	Sessions	Stage_Order	Conversion_Rate	Drop_Off_Rate
0	Browse	19992	0	100.00	0.00
1	Add to Cart	476	1	2.38	97.62
2	Checkout	63	2	0.32	86.76
3	Purchase	63	3	0.32	0.00

```
Revenue Analysis:
Total Revenue: $65108.00
Average Order Value: $1033.46
Total Orders: 63.0
```

Next steps: Generate code with `funnel_df` New interactive sheet

Visualization - Overall Funnel

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

# Create vis
fig=make_subplots(
    rows=2, cols=2,
    subplot_titles=('Funnel Conversion Rates', 'Stages to Stage Drop-off',
                    'Revenue by Funnel Stage', 'Session Duration by Stage'),
    specs=[[{"secondary_y": False}, {"secondary_y": False}],
           [{"secondary_y": False}, {"secondary_y": False}]]
)

#Funnel conversion rates
fig.add_trace(
    go.Bar(x=funnel_df['Stage'], y=funnel_df['Sessions'], # Changed 'Session' to 'Sessions'
           text=funnel_df['Sessions'], textposition='auto',
    name='Sessions', marker_color='lightblue'),
    row=1, col=1
)

# Drop-off rates
fig.add_trace(
    go.Scatter(
        x=funnel_df['Stage'],
        y=funnel_df['Drop_Off_Rate'],
        mode='lines+markers+text',
        text=funnel_df['Drop_Off_Rate'],
        textposition='top center',
        name='Drop-off Rate (%)',
        line=dict(color='red', width=3)
    ),
    row=1, col=2, secondary_y=False
)

# Revenue by stage
```

```
revenue_by_stage = session_summary.groupby('Max_Funnel_Stage')['Revenue'].sum().reset_index()
fig.add_trace(
    go.Bar(
        x=revenue_by_stage['Max_Funnel_Stage'],
        y=revenue_by_stage['Revenue'],
        text=[f'{x:.0f}' for x in revenue_by_stage['Revenue']],
        textposition='auto',
        name='Revenue',
        marker_color='green'
    ),
    row=2, col=1
)

# Session duration by stage
duration_by_stage = session_summary.groupby('Max_Funnel_Stage')['Session_Duration_Min'].mean().reset_index()
fig.add_trace(
    go.Bar(
        x=duration_by_stage['Max_Funnel_Stage'],
        y=duration_by_stage['Session_Duration_Min'],
        text=duration_by_stage['Session_Duration_Min'].round(2),
        textposition='auto',
        name='Avg Session Duration (min)',
        marker_color='orange'
    ),
    row=2, col=2 # Changed row from 3 to 2, and col to 2
)
fig.update_layout(height=800, title_text="Comprehensive Funnel Analysis Dashboard", showlegend=False)
fig.show()
```

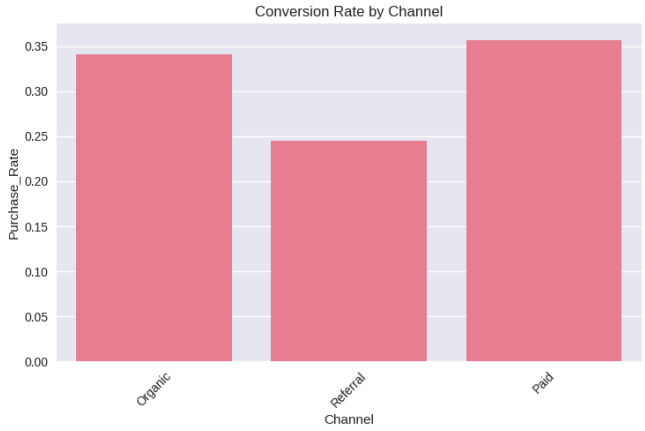
Channel Performance Analysis

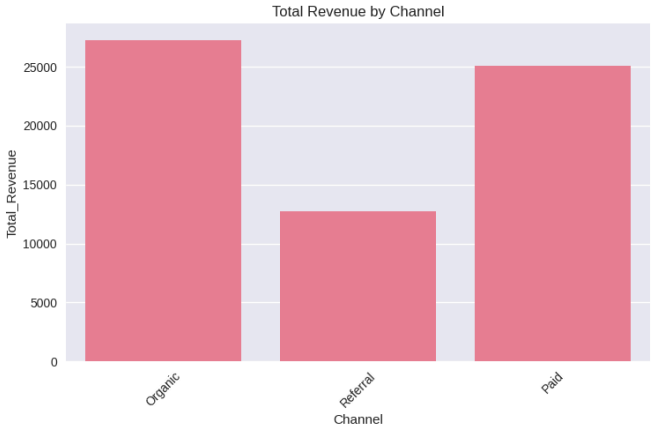
```
# Funnel analysis by channel
```

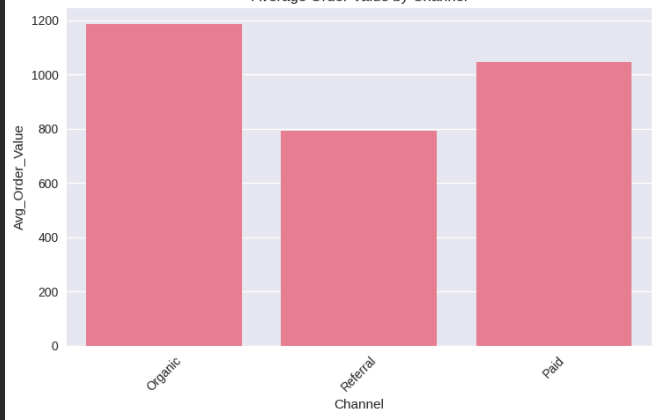
```
# Funnel analysis by channel
channel_funnel=[]
for channel in df['Channel'].unique():
  channel_sessions=session_summary[session_summary['Channel']==channel]
  total_sessions=len(channel_sessions)
  if total_sessions>0:
    channel_metrics={'Channel': channel, 'Total_Sessions': total_sessions}
    for i, stage in enumerate(funnel_stages):
      if i==0:
        count=total_sessions
      else:
        count=len(channel_sessions[channel_sessions['Max_Funnel_Stage'].isin(funnel_stages[i:])])
      channel_metrics[f'{stage}_Sessions']=count
      channel_metrics[f'{stage}_Rate']=(count/total_sessions*100)

    # Revenue metrics

    purchase_sessions=channel_sessions[channel_sessions['Max_Funnel_Stage']=='Purchase']
    channel_metrics['Total_Revenue']=purchase_sessions['Revenue'].sum()
    channel_metrics['Avg_Order_Value']=purchase_sessions['Revenue'].mean() if len(purchase_sessions)>0 else 0
    channel_metrics['Total_Orders']=(len(purchase_sessions)/total_sessions*100)
    channel_funnel.append(channel_metrics)
channel_df=pd.DataFrame(channel_funnel)
print("Performance Analysis: ")
display(channel_df.round(2))
#visualize channel performance
fig, ((ax1, ax2), (ax3, ax4)) =plt.subplots(2, 2, figsize=(15, 10))
# conversion rates by channel
sns.barplot(data=channel_df, x='Channel', y='Purchase_Rate', ax=ax1)
ax1.set_title('Conversion Rate by Channel')
ax1.tick_params(axis='x', rotation=45)
# Total revenue by  channel
sns.barplot(data=channel_df, x='Channel', y='Total_Revenue', ax=ax2)
ax2.set_title('Total Revenue by Channel')
ax2.tick_params(axis='x', rotation=45)

#AOV by channel
sns.barplot(data=channel_df, x='Channel', y='Avg_Order_Value', ax=ax3)
ax3.set_title('Average Order Value by Channel')
ax3.tick_params(axis='x', rotation=45)
#session distribution by channel
channel_df['Session_Percentage']=(channel_df['Total_Sessions']/channel_df['Total_Sessions'].sum()*100)
ax4.pie(channel_df['Session_Percentage'], labels=channel_df['Channel'], autopct='%1.1f%%')
ax4.set_title('Session Distribution by Channel')
plt.tight_layout()
plt.show()
```

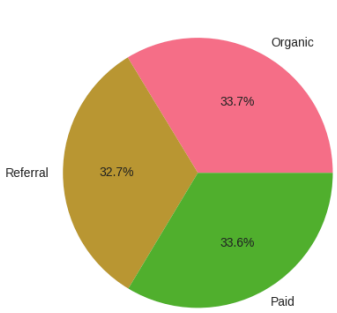
```
Performance Analysis:
```

	Channel	Total_Sessions	Browse_Sessions	Browse_Rate	Add to Cart_Sessions	Add to Cart_Rate	Checkout_Sessions	Checkout_Rate	Purchase_Se
0	Organic	6736	6736	100.0	178	2.64	23	0.34	
1	Referral	6536	6536	100.0	146	2.23	16	0.24	
2	Paid	6720	6720	100.0	152	2.26	24	0.36	









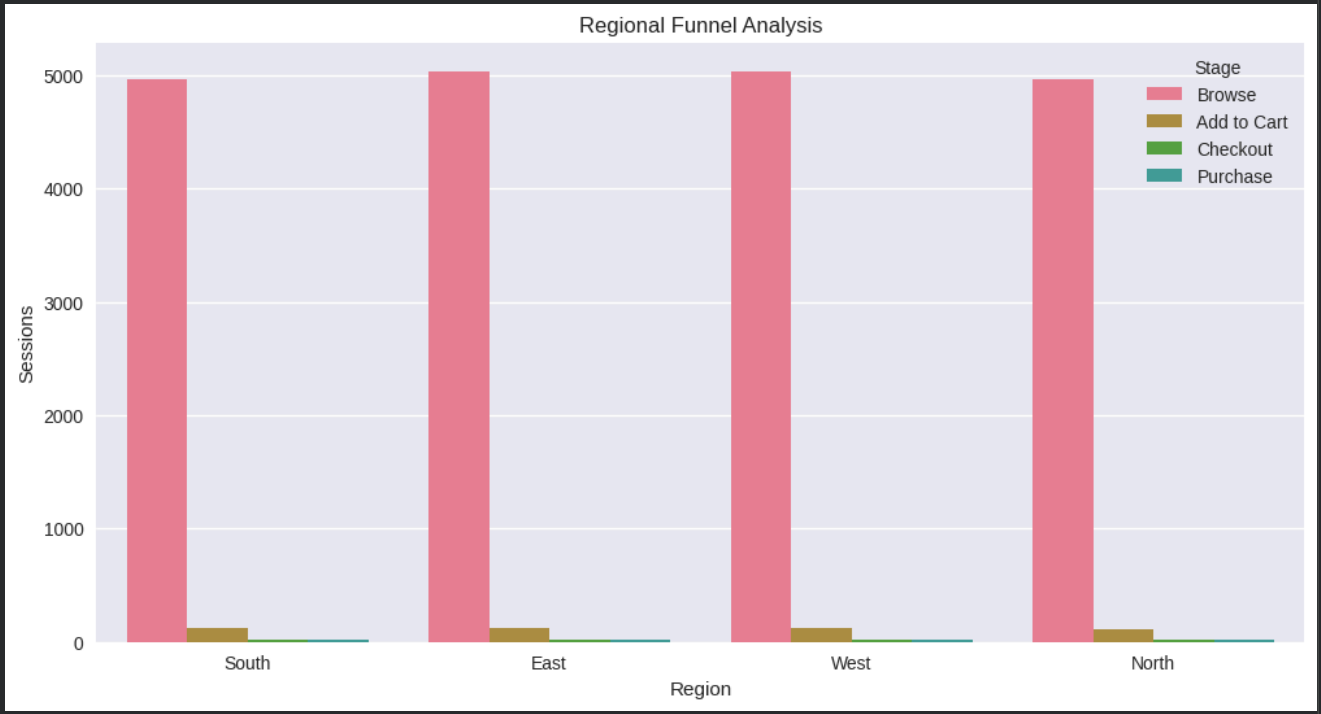
Regional Analysis

```
regional_analysis=session_summary.groupby('Region').agg(
    Total_Sessions=('Session_ID', 'count'),
    Total_Revenue=('Revenue', 'sum'),
    Avg_Session_Duration_Min=('Session_Duration_Min', 'mean')
)
# Add conversion rates by region
regional_conversion=session_summary[session_summary['Max_Funnel_Stage']=='Purchase'].groupby('Region').size()
regional_analysis['Converted_Sessions']=regional_conversion
regional_analysis['Conversion_Rate']=(regional_analysis['Converted_Sessions']/regional_analysis['Total_Sessions']*100)
regional_analysis['AOV']=(regional_analysis['Total_Revenue']/regional_analysis['Converted_Sessions']).round(2)
print("Regional Performance:")
display(regional_analysis)

#Regional funnel visualization
regional_funnel_data=[]
for region in df['Region'].unique():
  region_sessions=session_summary[session_summary['Region']==region]
  for stage in funnel_stages:
    if stage=='Browse':
      count=len(region_sessions)
    else:
      count=len(region_sessions[region_sessions['Max_Funnel_Stage'].isin(funnel_stages[funnel_stages.index(stage):])])
    regional_funnel_data.append({'Region': region, 'Stage': stage, 'Sessions': count})
regional_funnel_df=pd.DataFrame(regional_funnel_data)
```

```
plt.figure(figsize=(12, 6))
sns.barplot(data=regional_funnel_df, x='Region', y='Sessions', hue='Stage')
plt.title('Regional Funnel Analysis')
plt.xlabel('Region')
plt.ylabel('Sessions')
plt.show()
```

```
Regional Performance:
```

Region	Total_Sessions	Total_Revenue	Avg_Session_Duration_Min	Converted_Sessions	Conversion_Rate	AOV
East	5032	17577	0.0	17	0.34	1033.94
North	4963	16676	0.0	16	0.32	1042.25
South	4962	11796	0.0	14	0.28	842.57
West	5035	19059	0.0	16	0.32	1191.19



Next steps: Generate code with regional_analysis | New interactive sheet

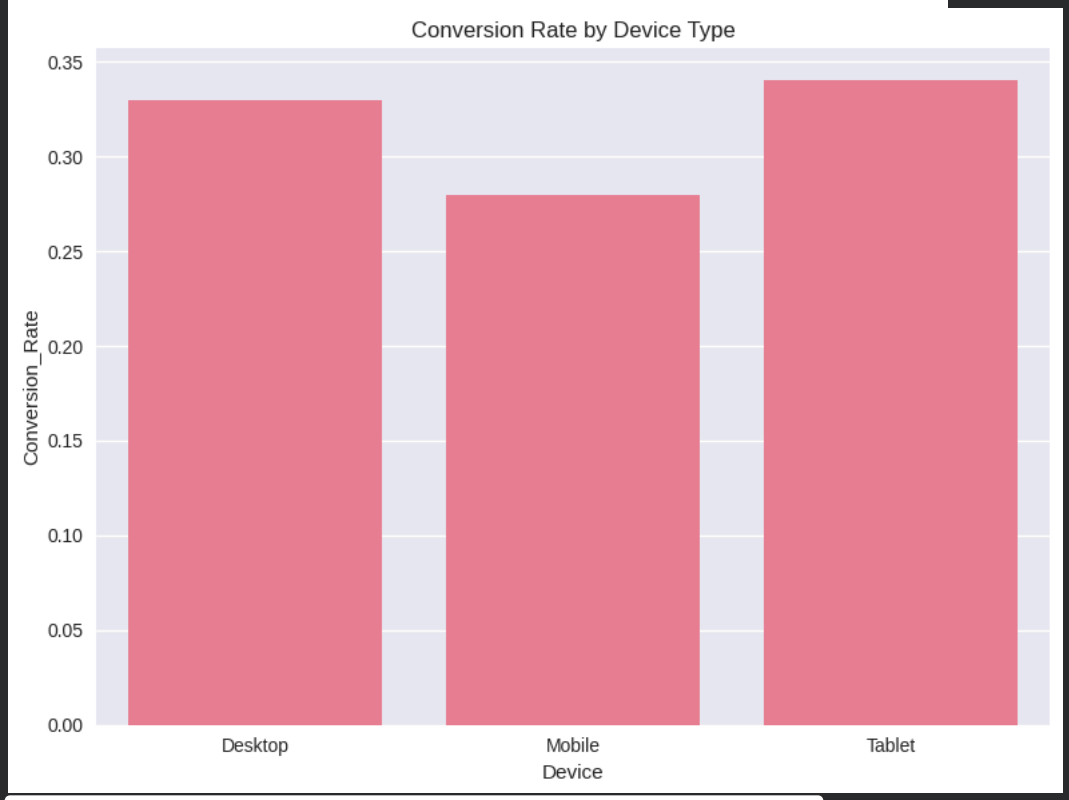
Device and Product Category Analysis

```
# Device performance
device_analysis = session_summary.groupby('Device').agg({
    'Session_ID': 'count',
    'Revenue': 'sum',
    'Session_Duration_Min': 'mean',
    'Max_Funnel_Stage': lambda x: (x == 'Purchase').sum()
}).rename(columns={'Session_ID': 'Total_Sessions', 'Max_Funnel_Stage': 'Purchases'})

device_analysis['Conversion_Rate'] = (device_analysis['Purchases'] / device_analysis['Total_Sessions'] * 100).round(2)
device_analysis['AOV'] = (device_analysis['Revenue'] / device_analysis['Purchases']).round(2)

print("# Device Performance")
display(device_analysis)

# Combined visualization
fig, ax1 = plt.subplots(1, 1, figsize=(8, 6))

#Device performance
sns.barplot(data=device_analysis.reset_index(), x='Device', y='Conversion_Rate', ax=ax1)
ax1.set_title('Conversion Rate by Device Type')
```

```
plt.tight_layout()
plt.show()
```

```
# Device Performance
```

Device	Total_Sessions	Revenue	Session_Duration_Min	Purchases	Conversion_Rate	AOV
Desktop	6701	26115	0.0	22	0.33	1187.05
Mobile	6738	20616	0.0	19	0.28	1085.05
Tablet	6553	18377	0.0	22	0.34	835.32



Next steps: Generate code with device_analysis | New interactive sheet

Time-Based Analysis

```
#Time-Based Analysis

# Extract Date and Hour from Timestamp for grouping
df['Date'] = df['Timestamp'].dt.date
df['Hour'] = df['Timestamp'].dt.hour

# Daily Trends
daily_metrics = (
    df.groupby('Date')
      .agg({
          'Session_ID': 'nunique',
          'User_ID': 'nunique',
          'Revenue': 'sum'
    })
    .rename(columns={
          'Session_ID': 'Daily_Sessions',
          'User_ID': 'Daily_Users'
     })
)

#  Add Conversion Rates (Daily)
# Extract date from datetime for grouping
daily_conversions=(
    session_summary[session_summary['Max_Funnel_Stage'] == 'Purchase']
    .groupby(session_summary['Session_Start'].dt.date)
```

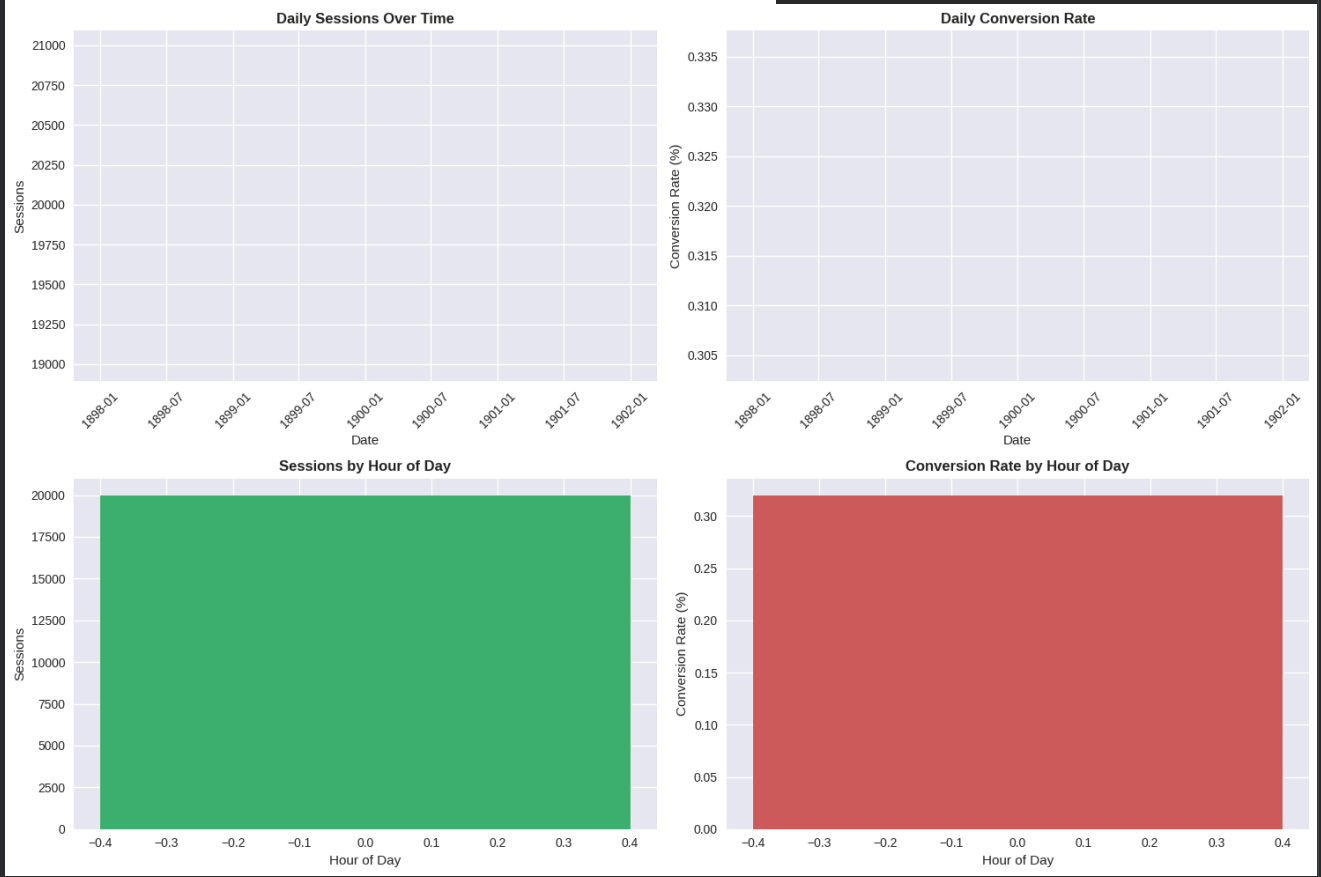
```
  .size()
)
daily_metrics['Daily_Conversions']=daily_conversions
daily_metrics['Daily_Conversion_Rate'] = (
    daily_metrics['Daily_Conversions']/daily_metrics['Daily_Sessions'] * 100
).round(2)

print("Daily Performance Trends")
display(daily_metrics.tail(10))

# Hourly Patterns
hourly_sessions = (
    df.groupby('Hour')
      .agg({
          'Session_ID': 'nunique',
          'Revenue': 'sum'
      })
      .rename(columns={'Session_ID': 'Hourly_Sessions'})
)

hourly_conversions =session_summary[
    session_summary['Max_Funnel_Stage'] == 'Purchase'
].copy()

hourly_conversions['Hour']= hourly_conversions['Session_Start'].dt.hour
hourly_conversion_counts= hourly_conversions.groupby('Hour').size()

hourly_sessions['Hourly_Conversions'] =hourly_conversion_counts

hourly_sessions['Hourly_Conversion_Rate'] = (
    hourly_sessions['Hourly_Conversions'] / hourly_sessions['Hourly_Sessions'] * 100
).round(2)


print(" Hourly Performance Trends")
display(hourly_sessions.tail(10))
# Visualization of Time-Based Metrics
import matplotlib.pyplot as plt

# Create a 2x2 grid of subplots
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(15, 10))

# Daily Sessions
ax1.plot(daily_metrics.index, daily_metrics['Daily_Sessions'], color='steelblue', linewidth=2)
ax1.set_title('Daily Sessions Over Time', fontsize=12, fontweight='bold')
ax1.set_xlabel('Date')
ax1.set_ylabel('Sessions')
ax1.tick_params(axis='x', rotation=45)

# --- Daily Conversion Rate ---
ax2.plot(daily_metrics.index, daily_metrics['Daily_Conversion_Rate'], color='darkorange', linewidth=2)
ax2.set_title('Daily Conversion Rate', fontsize=12, fontweight='bold')
ax2.set_xlabel('Date')
ax2.set_ylabel('Conversion Rate (%)')
ax2.tick_params(axis='x', rotation=45)

#Hourly Session Pattern
ax3.bar(hourly_sessions.index, hourly_sessions['Hourly_Sessions'], color='mediumseagreen')
ax3.set_title('Sessions by Hour of Day', fontsize=12, fontweight='bold')
ax3.set_xlabel('Hour of Day')
ax3.set_ylabel('Sessions')

#Hourly Conversion Rate
ax4.bar(hourly_sessions.index, hourly_sessions['Hourly_Conversion_Rate'], color='indianred')
ax4.set_title('Conversion Rate by Hour of Day', fontsize=12, fontweight='bold')
ax4.set_xlabel('Hour of Day')
ax4.set_ylabel('Conversion Rate (%)')

# Adjust layout for better spacing
plt.tight_layout()
plt.show()
```

```
Daily Performance Trends
```

Date	Daily_Sessions	Daily_Users	Revenue	Daily_Conversions	Daily_Conversion_Rate
1900-01-01	19992	10000	65108	63	0.32

```
Hourly Performance Trends
```

Hour	Hourly_Sessions	Revenue	Hourly_Conversions	Hourly_Conversion_Rate
0	19992	65108	63	0.32



Advance Funnel Metrics and KPIs

```
print("🔸 KEY PERFORMANCE INDICATORS (KPIs)")
print("=" * 50)

#Overall KPIs
total_sessions = len(session_summary)
total_revenue = session_summary['Revenue'].sum()
total_orders = len(session_summary[session_summary['Max_Funnel_Stage'] == 'Purchase'])
overall_conversion_rate = (total_orders / total_sessions) * 100

print(f"🔥Overall Conversion Rate: {overall_conversion_rate:.2f}%")
print(f"💰Total Revenue: ${total_revenue:,.2f}")
print(f"📊Average Order Value: ${(total_revenue / total_orders):,.2f}")
print(f"🧮Total Sessions: {total_sessions:,}")
print(f"🛒Total Orders: {total_orders:,}")

# Funnel Efficiency Metrics
browse_to_cart = (funnel_df.iloc[1]['Sessions'] / funnel_df.iloc[0]['Sessions']) * 100
cart_to_checkout = (funnel_df.iloc[2]['Sessions'] / funnel_df.iloc[1]['Sessions']) * 100
checkout_to_purchase = (funnel_df.iloc[3]['Sessions'] / funnel_df.iloc[2]['Sessions']) * 100
```

```
print("\n📈 Stage-to-Stage Conversion Rates:")
print(f"➡️ Browse → Add to Cart: {browse_to_cart:.2f}%")
print(f"➡️ Add to Cart → Checkout: {cart_to_checkout:.2f}%")
print(f"➡️ Checkout → Purchase: {checkout_to_purchase:.2f}%")

# Revenue per Session at Each Stage
revenue_per_browse = total_revenue / funnel_df.iloc[0]['Sessions']
revenue_per_cart = total_revenue / funnel_df.iloc[1]['Sessions']
revenue_per_checkout = total_revenue / funnel_df.iloc[2]['Sessions']

print("\n💵 Revenue per Session by Stage:")
print(f"🗂️ Browse Stage: ${revenue_per_browse:,.2f}")
print(f"🛒 Add to Cart Stage: ${revenue_per_cart:,.2f}")
print(f"💳 Checkout Stage: ${revenue_per_checkout:,.2f}")
# Bounce Rate Analysis
bounce_sessions = session_summary[session_summary['Bounce_Flag'] == 'Yes']
bounce_rate = (len(bounce_sessions) / total_sessions) * 100
print(f"\n📉 Bounce Rate: {bounce_rate:.2f}%")

#  SessionDurationAnalysis
avg_session_duration = session_summary['Session_Duration_Min'].mean()
print(f"⏱️ Average Session Duration: {avg_session_duration:.2f} minutes")
```

```
🔸 KEY PERFORMANCE INDICATORS (KPIs)
==================================================
🔥Overall Conversion Rate: 0.32%
💰Total Revenue: $65,108.00
📊Average Order Value: $1,033.46
🧮Total Sessions: 19,992
🛒Total Orders: 63

📈Stage-to-Stage Conversion Rates:
➡️Browse → Add to Cart: 2.38%
➡️ Add to Cart → Checkout: 13.24%
➡️ Checkout → Purchase: 100.00%

💵 Revenue per Session by Stage:
🗂️Browse Stage: $3.26
🛒 Add to Cart Stage: $136.78
💳 Checkout Stage: $1,033.46

📉Bounce Rate: 0.00%
⏱️ Average Session Duration: 0.00 minutes
```

Strategic Recommendations

```
# Funnel Performance Insights

# Identify Biggest Drop-Off Points
max_dropoff_stage = funnel_df.loc[funnel_df['Drop_Off_Rate'].idxmax()]
print(f"💥 BIGGEST DROP-OFF: {max_dropoff_stage['Stage']} stage with {max_dropoff_stage['Drop_Off_Rate']:.2f}% drop-of

# Best Performing Channel
best_channel = channel_df.loc[channel_df['Purchase_Rate'].idxmax()]
print(f"🏆 BEST PERFORMING CHANNEL: {best_channel['Channel']} with {best_channel['Purchase_Rate']:.2f}% conversion")

# BestPerforming Region
best_region_name = "N/A (No conversions)"
best_region_conversion = 0.0
if regional_analysis['Converted_Sessions'].sum() > 0:
    best_region_data = regional_analysis.loc[regional_analysis['Conversion_Rate'].idxmax()]
    best_region_name = best_region_data.name
    best_region_conversion = best_region_data['Conversion_Rate']
print(f"🌍 BEST PERFORMING REGION: {best_region_name} with {best_region_conversion:.2f}% conversion")


# Best Performing Product Category (Removed as 'Product_Category' column does not exist)
best_category_name = "N/A (No conversions)"
best_category_conversion = 0.0
# if product_analysis['Purchases'].sum() > 0:
#     best_category_data = product_analysis.loc[product_analysis['Conversion_Rate'].idxmax()]
#     best_category_name = best_category_data.name
#     best_category_conversion = best_category_data['Conversion_Rate']
print(f"🗂️ BEST PERFORMING CATEGORY: {best_category_name} with {best_category_conversion:.2f}% conversion")
```

```
# RecommendedActions
print("\n🔧 RECOMMENDED ACTIONS:")
print(f"1️⃣ Address {max_dropoff_stage['Stage']} stage drop-off through UX improvements.")
print(f"2️⃣ Allocate more budget to {best_channel['Channel']} channel.")
if best_region_name != "N/A (No conversions)":
    print(f"3️⃣ Replicate {best_region_name} region strategies in underperforming regions.")
else:
    print("3️⃣ Cannot make region-specific recommendations due to no conversions.")
# if best_category_name != "N/A (No conversions)": # Removed as 'Product_Category' column does not exist
#     print(f"4️⃣ Promote {best_category_name} category to improve overall conversion.")
# else:
print("4️⃣ Cannot make product category-specific recommendations due to no conversions.")
print(f"5️⃣ Focus on cart abandonment recovery for {funnel_df.iloc[2]['Drop_Off_Rate']:.2f}% of users.")

# Calculate PotentialRevenueOpportunities
# cart_abandonment_opportunity_value = funnel_df.iloc[2]['Sessions'] * product_analysis['AOV'].mean() # Removed as 'P
cart_abandonment_opportunity_value = 0 # Set to 0 or handle differently if AOV can be calculated from other metrics
if pd.isna(cart_abandonment_opportunity_value) or cart_abandonment_opportunity_value == 0:
    print("\n💰 REVENUE OPPORTUNITY: Not calculable due to no purchase data or zero potential revenue.")
else:
    print("\n💰 REVENUE OPPORTUNITY:")
    print(f"Cart abandonment recovery: ${cart_abandonment_opportunity_value:,.2f} potential revenue")
```

```
💥BIGGEST DROP-OFF: Add to Cart stage with 97.62% drop-off
🏆BEST PERFORMING CHANNEL: Paid with 0.36% conversion
🌍BEST PERFORMING REGION: East with 0.34% conversion
🗂️ BEST PERFORMING CATEGORY: N/A (No conversions) with 0.00% conversion

🔧 RECOMMENDED ACTIONS:
1️⃣ Address Add to Cart stage drop-off through UX improvements.
2️⃣ Allocate more budget to Paid channel.
3️⃣ Replicate East region strategies in underperforming regions.
4️⃣ Cannot make product category-specific recommendations due to no conversions.
5️⃣ Focus on cart abandonment recovery for 86.76% of users.

💰 REVENUE OPPORTUNITY: Not calculable due to no purchase data or zero potential revenue.
```

Result

```
#Create Comprehensive Report
report_data = {
    'Overall_Funnel': funnel_df,
    'Channel_Performance': channel_df,
    'Regional_Analysis': regional_analysis,
    'Device_Performance': device_analysis,
    'Daily_Trends': daily_metrics,
    'Hourly_Patterns': hourly_sessions
}

# Export to Excel for Corporate Reporting
with pd.ExcelWriter('funnel_analysis_report.xlsx') as writer:
    for sheet_name, data in report_data.items():
        data.to_excel(writer, sheet_name=sheet_name, index=False)

print("Analysis complete! Report exported to 'funnel_analysis_report.xlsx'")

# Save Key Visualizations
plt.figure(figsize=(10, 6))
sns.barplot(data=funnel_df, x='Stage', y='Conversion_Rate', hue='Stage', palette='viridis', legend=False)
plt.title('Overall Funnel Conversion Rates', fontsize=14, fontweight='bold')
plt.xlabel('Stage')
plt.ylabel('Conversion Rate (%)')
```